

Step 3 — Retry and Fallback for the Drafting Call — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real `.with_retry()` / `.with_fallbacks()` calls), **Advanced** (the design question about what counts as an acceptable degraded answer). Read Basic first even if you've handled retries before — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Retry vs. fallback are two different questions](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Retry vs. fallback are two different questions

Basic Version

A **retry** answers "should I just try the exact same thing again?" — good for a hiccup that's likely to clear up on its own, like a timeout or a rate limit. A **fallback** answers "the retries didn't work — now what?" — it's a genuinely different path, not just trying the same thing one more time.

Things to use: - `.with_retry(...)` on your model, for transient failures only. - `.with_fallbacks(...)` on your chain, for when retries run out. - A second, simpler chain (a cheaper model, or no model call at all) to act as the fallback.

Intermediate Version

`model.with_retry(retry_if_exception_type=(SomeError, AnotherError), stop_after_attempt=3, wait_exponential_jitter=True)` wraps your model so LangChain automatically retries the call, with a growing (and slightly randomized) wait between attempts, but **only** for the exception types you named. This matters: if you don't scope `retry_if_exception_type`, LangChain will happily retry an error that was never going to succeed no matter how many times you ask — like a validation error from a badly-formed prompt.

`chain.with_fallbacks([fallback_chain])` returns a new chain that runs `chain` normally, and if it raises an exception (after any retries are exhausted), runs `fallback_chain` instead, with the exact same input. The fallback chain doesn't have to be another LLM call at all — a fixed, canned-response chain built with `RunnableLambda` is a completely valid fallback, and often the more honest one.

The exact pieces: - `from openai import RateLimitError, APITimeoutError, APIConnectionError` — real exception types worth scoping retries to. - `.with_retry(retry_if_exception_type=(RateLimitError, APITimeoutError, APIConnectionError), stop_after_attempt=3)` - `.with_fallbacks([fallback_chain])` on the whole drafting chain (`prompt | resilient_model | parser`), not just on the bare model.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the real question this step is testing: a fallback response is, by definition, worse than what you wanted. For a general "do you have a referral program" ticket, a canned "thanks, we'll follow up shortly" reply costs almost nothing to send if the real drafting call fails. For a billing ticket about an actual double-charge, that same canned reply is arguably worse than sending nothing at all — it tells the customer their real financial problem was heard and is being handled, when actually nothing happened yet.

There's no single correct answer here, but there is a wrong one: not deciding on purpose. A pipeline that treats every category's fallback the same way, without ever asking "is a degraded answer actually okay here," has made a real decision by accident. Write your actual answer down, as a comment, before you move on — even if your answer is "yes, the canned reply is fine everywhere, because it's clearly generic and a human will still review every billing ticket regardless."

Difference between Basic, Intermediate, and Advanced: Basic separates "try again" from "now what." Intermediate shows the real `.with_retry()` and `.with_fallbacks()` calls, scoped correctly. Advanced isn't about the API at all — it's the honest design question of when a worse answer is acceptable to actually send, which no library call can decide for you.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
build the model, wrap it with retry (only for timeouts/rate limits, try 3 times)
build a fallback chain (a canned reply, or a cheaper model)

drafting_chain = prompt + retrying_model + parser
safe_drafting_chain = drafting_chain, but fall back to fallback_chain if it still

test: force a failure on purpose -> confirm the fallback actually answers
test: a normal call -> confirm the fallback is never touched
```

Intermediate Version

```
# drafting.py
from langchain_openai import ChatOpenAI
from openai import RateLimitError, APITimeoutError, APIConnectionError
from config import MODEL_NAME, MODEL_TEMPERATURE

def build_resilient_model():
    model = ChatOpenAI(model=MODEL_NAME, temperature=MODEL_TEMPERATURE)
    return model.with_retry(
        retry_if_exception_type=(RateLimitError, APITimeoutError, APIConnectionError),
        stop_after_attempt=3,
        wait_exponential_jitter=True,
    )
```

Now write `fallback.py`'s `build_fallback_chain()` yourself (start with the simpler canned-template version) and wire `.with_fallbacks([...])` onto one handler chain in `handlers.py`, before checking the Advanced version below.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
# fallback.py
# Design decision: a canned fallback reply is acceptable for general and
# technical tickets (low stakes if slightly generic), but NOT acceptable
# on its own for billing tickets involving a real charge dispute -- those
# should still get a reply, but flagged needs_human_review=True so a
# person checks it before it's treated as resolved. (Fill in your own
# answer here -- this is a real judgment call, not a fact to look up.)

from langchain_core.runnables import RunnableLambda

def canned_response(inputs):
    return (
        "Thanks for reaching out. We've received your message and a "
        "member of our team will follow up with you shortly."
    )

def build_fallback_chain():
    return RunnableLambda(canned_response)
```

```
# handlers.py -- wiring retry + fallback into billing_chain
from drafting import build_resilient_model
from fallback import build_fallback_chain

resilient_model = build_resilient_model()
fallback_chain = build_fallback_chain()

billing_chain = (
    (research_chain | billing_prompt | resilient_model | StrOutputParser())
    .with_fallbacks([fallback_chain])
)
```

Now write the forced-failure test yourself: swap in a model name that doesn't exist, or raise a fake exception from a test double, and confirm `billing_chain.invoke(...)` still returns *something* (the canned response) instead of crashing. Compare your finished version against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic is the plain "try again, then fall back" shape. Intermediate is the real `.with_retry()` and `.with_fallbacks()` wiring, scoped to real transient exception types. Advanced adds the one thing no library call can add for you: an actual, written-down decision about when a degraded answer is good enough to send.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)